Training a Native Sinhala AI Model

https://gemini.google.com/app/7337214e284b98c9

User prompt: i want to develop and train a ai model which is capable of understanding and responding to native sinhala language.

Response: Building a native Sinhala conversational AI from the ground up is rarely done by training a foundation model from scratch—which requires billions of tokens and massive GPU clusters. Instead, the most effective engineering approach is **Continued Pre-Training (CPT)** and **Supervised Fine-Tuning (SFT)** on top of an existing open-weights multilingual model using Parameter-Efficient Fine-Tuning (PEFT / QLoRA).

1. Selecting the Base Architecture

Because Sinhala is a morphologically rich, low-resource language with a unique Brahmic script, your biggest bottleneck in standard LLMs is **tokenizer fertility** (how many tokens it takes to represent a single Sinhala word). If a tokenizer splits a single Sinhala word into 6–10 byte-level fallback tokens, inference becomes slow and context windows shrink dramatically.

Base Model	Parameter Sizes	Sinhala Suitability & Tokenizer Behavior
Gemma 2 / Gemma 3	2B, 9B, 27B	**Top Recommendation.** Massive vocabulary (256k tokens) with strong native representation of South Asian scripts. Tokenizes Sinhala cleanly without heavy byte-fallback.
Llama 3.1 / 3.2 (or SinLlama)	3B, 8B	128k vocabulary. Community adaptations like **SinLlama** (Llama-3-8B extended on 300M+ Sinhala tokens) already exist and drastically outperform base Llama on Sinhala tasks.
Qwen 2.5	3B, 7B, 14B	151k vocabulary. Exceptional multilingual reasoning and strong alignment capabilities when fine-tuned with LoRA adapters.
Aya Expanse (Cohere)	8B, 32B	Explicitly engineered for multilingual breadth and under-represented languages; strong zero-shot baseline.

2. Dataset Strategy & Curation

To make the model both *understand* native grammar (both formal written *Likhitha* and spoken colloquial *Kathitha*) and *respond* helpfully, you need two distinct data layers:

Unsupervised Corpora (For Language & Cultural Grounding)

If your base model struggles with fluent Sinhala syntax, run a stage of Continued Pre-Training on raw text:

- **Hugging Face `sinhala-nlp` Hub:** Pull the **NSINA** (Large-scale Sinhala News Corpus) and the **Sinhala-1.5B-corpus**.
- **Cleaned Web Crawls:** Use **CulturaX** (Sinhala subset) and **MADLAD-400** (specifically the `polyglots/MADLAD_CulturaX_cleaned` dataset, which contains ~10.7M cleaned Sinhala sentences / 304M tokens).
- **Custom Scraping:** Scrape local news portals, Wikipedia (`si.wikipedia.org`), blogs, and public forums to capture modern colloquial phrasing.

Instruction & Chat Datasets (For SFT / Conversational Ability)

A base model only predicts the next word; to make it an assistant, you need `{instruction, input, output}` or multi-turn ShareGPT-style JSONL pairs:

- **Synthetic Data Generation (Distillation):** Use high-frontier commercial APIs (like Gemini 2.5 Pro or Claude) to generate or translate high-quality instruction datasets (like UltraChat, OpenAssistant, or Dolly) into natural, idiomatic Sinhala. Prompt the teacher model strictly to avoid robotic, literal word-for-word translations.
- **Diglossia Handling:** Ensure your instruction set includes both formal Sinhala prompts and casual spoken-style Sinhala (and optionally **Singlish** transliteration, such as `sinhala-nlp/translit`, if your users type Sinhala using the English alphabet).

3. End-to-End Training Pipeline

1. **Preprocess & Normalize Unicode Text**

   Critical: Inconsistent Unicode breaks tokenization

   Clean your raw and instruction datasets in Python. Normalize all text to **Unicode NFC**, strip invisible zero-width joiners (`\u200d` / `\u200c`) unless strictly required for specific conjunct characters (*Yansaya*, *Rakaransaya*, *Repaya*), and deduplicate using MinHash.

2. **Evaluate (and Optionally Extend) the Tokenizer**

   Test how your chosen base model tokenizes a sample paragraph of Sinhala. If you choose Gemma or SinLlama, you can skip tokenizer surgery. If you use a model with poor Sinhala vocabulary, train a custom SentencePiece tokenizer on your Sinhala corpus, merge the new tokens into the base model, and resize the `embed_tokens` and `lm_head` layers.

3. **Continued Pre-Training (Optional - Domain/Language Adaptation)**

   If starting from a raw base model (or if you added new tokens), train the model on your raw Sinhala text corpus using **Unsloth** or **Axolotl**. Unfreeze `embed_tokens` and `lm_head` along with LoRA adapters on all linear layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`).

4 **Supervised Fine-Tuning (SFT) with QLoRA**

Format your conversational Sinhala dataset into the model's native chat template (e.g., ChatML or Gemma `<start_of_turn>` tags). Train using 4-bit **QLoRA** (Rank $r = 32$ or 64, $\alpha = 64$, learning rate 2×10^{-4} with cosine decay) for 2 to 3 epochs.

5 **Preference Alignment (DPO / ORPO)**

Once the model answers instructions in Sinhala, run **Direct Preference Optimization (DPO)** using a small dataset of `(prompt, chosen_natural_sinhala, rejected_awkward_sinhala)` pairs. This eliminates "translationese" and teaches the model natural Sri Lankan phrasing.

6 **Quantization & Deployment**

Merge your LoRA adapters back into the 16-bit base weights, then export to **GGUF** (for `llama.cpp` / Ollama) or **AWQ** (for high-throughput `vLLM` production serving on a GPU VPS).

4. Recommended Tech Stack & Hardware Requirements

You do not need an enterprise cluster to fine-tune an 8B or 9B Sinhala model:

- **Training Framework: Unsloth** (reduces VRAM usage by ~60% and speeds up training 2x) or **LLaMA-Factory** (provides both CLI and a clean Web UI for managing SFT and DPO pipelines).
- **VRAM & Compute:**
  - **QLoRA SFT (8B–9B model):** 1x NVIDIA RTX 3090 / 4090 (24GB VRAM) or an A10G/L4 on cloud providers (RunPod, vast.ai, or Lambda Labs).
  - **Continued Pre-Training:** 1x A100 (40GB/80GB) is recommended for larger batch sizes and longer context lengths.
- **Evaluation Benchmarks:** Test your checkpoints against **SiMTEB**, Sinhala sentiment/classification splits, and LLM-as-a-judge evaluations for grammatical fluency.

Sources

1. Sinhala-NLP/NSINA: A {N}ews Corpus for {Sin}hal{a} - GitHub

---

User prompt: is this a novelty?

Response: Simply "having an AI that understands and speaks Sinhala" is **no longer a novelty** on its own, because frontier commercial models (like Gemini, Claude, and GPT) and foundational research models like **SinLlama** (released by researchers from the University of Moratuwa in 2025) have already proven that LLMs can process Sinhala text.

However, **how you specialize, align, or deploy it** still holds massive novelty—both for academic research (like a university final-year research project or IEEE/ACL publication) and for commercial products in Sri Lanka.

1. What is Already Done (Not Novel)

If you propose any of the following as a standalone research contribution, reviewers will likely point to existing work:

- **Base Continual Pre-Training & Tokenizer Extension:** The **SinLlama** project (`polyglots/SinLlama_v01`) already extended the Llama-3-8B tokenizer to 139,336 tokens and trained it on 303.9M Sinhala tokens (10.7M sentences from MADLAD-400 and CulturaX).
- **Basic Classification Fine-Tuning:** Fine-tuning encoders (like SinBERT or XLM-R) or decoders on standard Sinhala sentiment analysis, basic news categorization, or simple machine-translated Alpaca datasets is well-trodden territory.
- **Generic API Wrappers:** Building a chat interface that just routes Sinhala prompts to a commercial API.

2. Where the Real Novelty Lies (Open Gaps)

Even though **SinLlama** exists, its authors explicitly note that it is a **base completion model, not a conversational chat/instruct model**—and it was only evaluated on text classification, not open-ended dialogue or reasoning. That leaves several high-impact gaps wide open:

Area of Novelty	Why It Is Currently Broken / Missing	How You Make It Novel
1. True Conversational Alignment (SFT + DPO for Spoken Sinhala)	Existing open-weights Sinhala models suffer from severe **"translationese"**—they output stiff, textbook *Likhitha Bhashawa* (written grammar) because their training sets were machine-translated from English datasets.	Building the first open-source **Preference-Aligned (DPO/ORPO) Sinhala Instruct Model** that responds naturally in colloquial *Kathitha Bhashawa* (spoken grammar) without sounding like Google Translate.
2. Singlish & Code-Mixing Native Fluency	Real Sri Lankan users rarely type pure Sinhala Unicode in chat; they mix English technical terms with Romanized Sinhala ("Singlish" like *"Mata meka explain karanna"*).	Training a model specifically on **tri-modal Sinhala input** (Pure Sinhala Unicode + Romanized Singlish + Sinhala-English code-switching) so it understands how locals actually message.
3. Sri Lankan Cultural & Factual Grounding (Hallucination Mitigation)	Base multilingual models know the Sinhala script, but they hallucinate wildly on Sri Lankan Penal Code, GCE A/L curricula, local banking procedures, and historical/cultural nuances.	Creating a **Domain-Specific Sinhala Model** (e.g., Sri Lankan Legal, Public Services, Education, or Healthcare Assistant) backed by a native Sinhala RAG benchmark.

Area of Novelty	Why It Is Currently Broken / Missing	How You Make It Novel
4. Small, Edge-Deployable Sinhala Models (SLMs)	Running an 8B or 27B parameter model requires expensive GPU hosting, making it unviable for local SMEs or offline apps.	Distilling Sinhala conversational fluency into a **1B to 3B parameter Small Language Model** (using Qwen-2.5-1.5B, Llama-3.2-3B, or Gemma-2-2B) that runs offline on mobile devices or cheap CPU VPS instances.
5. Generative Sinhala Evaluation Benchmarks	Existing Sinhala NLP benchmarks only test classification (sentiment, news topic, POS tagging). There is no standard benchmark for **generative conversational quality**.	Authoring a **Sinhala MT-Bench or MMLU-style evaluation suite** that tests reasoning, diglossia accuracy, and cultural alignment.

3. How to Position Your Project

- **If this is for Academic Research / Publication:** Do not pitch the project as *"Building a Sinhala AI."* Pitch a specific linguistic or architectural bottleneck—for example: *"Mitigating Diglossic Mismatch and Translationese in Low-Resource Sinhala Conversational LLMs via Direct Preference Optimization (DPO)"* or *"Cross-Script Code-Switching Adaptation for Romanized and Native Sinhala Dialogue Systems."*
- **If this is for a Commercial Product / Startup:** Your novelty isn't beating Gemini Pro on general trivia; your novelty is **cost, privacy, and local UX**—such as an on-premise/VPS-hosted Sinhala customer support agent for local businesses that fluently handles voice notes, Singlish WhatsApp messages, and local business workflows at a fraction of commercial API costs.

Sources

1. SAWithanage/SinLlama-Llama-3-8B-Merged - Hugging Face
2. polyglots/SinLlama_v01 - Hugging Face

User prompt: can we use more smaller model?

Response: Yes—moving down to a **Small Language Model (SLM)** in the **1B to 4B parameter range** (or even sub-1B like 500M) is not only possible, it is actually one of the strongest angles for both **academic novelty** and **practical local deployment**.

While SinLlama (8B) requires an expensive cloud GPU just to run, a 1B to 3B parameter Sinhala model can be **trained for free on a Google Colab T4 (16GB VRAM)** and deployed directly on a **standard CPU VPS, a desktop POS machine, or offline inside a mobile app** (at ~700MB to 1.8GB quantized in GGUF).

1. Best Sub-4B Base Models for Sinhala

When dropping below 4B parameters, the model has much less internal "capacity" to memorize broken byte-tokens. Therefore, choosing a base model with a **large, multilingual vocabulary** is critical:

Model Family	Sizes Available	Quantized Size (Q4_K_M)	Why Choose It for Sinhala?
Gemma 3 / Gemma 2	**1B, 2B, 4B**	~0.7 GB – 2.5 GB	**Best Choice.** Keeps the massive **256k/262k vocabulary** of its larger siblings. It already has native Sinhala character/subword tokens, so you don't always have to do complex tokenizer surgery to get fluent output.
Qwen 2.5 / Qwen 3	**0.5B, 1.5B, 3B**	~0.4 GB – 1.9 GB	Uses a 151k vocabulary with awesome reasoning density per parameter. The **1.5B and 3B** models punch way above their weight when fine-tuned on domain-specific instructions.
Llama 3.2	**1B, 3B**	~0.8 GB – 2.0 GB	Uses the 128k Llama-3 tokenizer. Like Llama-3-8B, it fragments Sinhala words heavily out-of-the-box, meaning **you must extend the tokenizer** (like SinLlama did) for it to perform well in Sinhala.

2. The Catch with Tiny Models (And How to Solve It)

When you shrink a model from 8B down to 1B or 2B, two things happen:

1. **World Knowledge Drops:** A 1.5B model cannot memorize all of Wikipedia. If you ask it obscure Sri Lankan historical facts without context, it will hallucinate more often than an 8B model.
2. **Language Capacity Shrinks:** A tiny model struggles if you ask it to be good at 100 languages *and* complex reasoning at the same time.

To make a 1B–3B Sinhala model perform like an 8B model, you use three specific techniques:

- **Knowledge Distillation (Teacher-Student Training):** Instead of training your 1.5B/2B model on raw web text, use a massive model (like Gemini 2.5 Pro) as the **Teacher** to generate 20,000–50,000 high-quality, step-by-step Sinhala conversational responses, and train your small **Student** model to mimic the Teacher.
- **Narrow the Task (Language + Task Specialization):** Don't expect a 1.5B model to write Python code, speak French, and recite history all at once. Train it specifically to be a **Sinhala/Singlish Conversational & RAG Assistant**. When you feed factual context into the prompt (via RAG or database lookup), a 1.5B model can summarize and respond in natural Sinhala almost as well as an 8B model.
- **Vocabulary Pruning / Grapheme Tokenization (High Research Value):** A 1B model with a 256k vocabulary spends a huge percentage of its total parameters just on the embedding layer (storing tokens for Chinese, Russian, Arabic, etc. that you will never use). Pruning unused foreign script tokens and keeping only **English + Sinhala** shrinks the model's RAM footprint by up to 35–40% and speeds up training.

3. Why a Smaller Model Is Actually *Better* for Your Project

- **Zero/Low Training Cost:** You can run **Full Fine-Tuning** (or high-rank LoRA) on a 1B–2B model using a single 16GB–24GB GPU in just a few hours, meaning you can iterate on your dataset 5x faster than someone training an 8B model.
- **Strong Research Benchmarking:** You can frame your research around **"Resource-Constrained / Edge-Native Sinhala SLMs"**—proving that a distilled, DPO-aligned **1.5B or 2B Sinhala model** can match or beat the unaligned **8B SinLlama** on conversational fluency and instruction-following (evaluated on benchmarks like **SinhalaMMLU** or LLM-as-a-Judge).
- **Real-World Deployability:** Once exported to **GGUF** or **MediaPipe / LiteRT**, your model can run inside a Flutter mobile app offline or on a $6/month CPU VPS with sub-second token latency.

User prompt: Show me how Gemma-2B/4B and Qwen-2.5-1.5B tokenize Sinhala text, and give me a quick script to test their token fertility on Google Colab.

Response: When comparing **Gemma (2B / 4B)** and **Qwen-2.5-1.5B** on Sinhala text, the fundamental difference comes down to **SentencePiece (Unigram/BPE with native Unicode subwords)** vs. **Byte-Level BPE**.

Because Sinhala characters sit in the `U+0D80–U+0DFF` Unicode block (where each character takes **3 bytes** in UTF-8), a tokenizer without dedicated Sinhala vocabulary will shatter a single Sinhala letter into 2 to 3 raw byte tokens.

1. Visual Breakdown: How They Tokenize Sinhala

Let’s take a natural 6-word Sinhala sentence:

> **Sentence:** "මම අද කොළඹ යනවා, ඔයා එනවද?" *(I am going to Colombo today, are you coming?)* **Word Count:** 6 words | **Character Count:** 26 characters

Metric / Behavior	Gemma 2 (2B) / Gemma 3 (1B/4B)	Qwen-2.5-1.5B (Base)	SinLlama / Custom Extended Qwen
Vocabulary Size	256,000 (Gemma 2) / 262,144 (Gemma 3)	151,643	~139,000 – 165,000 (Extended)

Metric / Behavior	Gemma 2 (2B) / Gemma 3 (1B/4B)	Qwen-2.5-1.5B (Base)	SinLlama / Custom Extended Qwen
Tokenization Mechanism	**Native Subwords & Graphemes:** Preserves whole common words (මම, අද) and splits longer words cleanly along syllabic/morphological boundaries (කොළඹ, යන, වා).	**Heavy Byte-Level Fallback:** Splits almost every Sinhala character or vowel sign (*pilla*) into individual UTF-8 bytes (`\xe0\xb6...`), causing massive token inflation.	**Whole Words & Morphemes:** Recognizes full Sinhala roots and suffixes (යනවා, එනවද) as single IDs.
Token Output (Conceptual)	`['මම', ' අද', ' කො', 'ළඹ', ' යන', 'වා', ',', ' ඔ', 'යා', ' එන', 'වද', '?']`	`['\xe0\xb6\xb8', '\xe0\xb6\xb8', ' ', '\xe0\xb6\x85', ...]` *(35+ broken byte fragments)*	`['මම', ' අද', ' කොළඹ', ' යනවා', ',', ' ඔයා', ' එනවද', '?']`
Estimated Token Count	**~11 – 14 tokens**	**~38 – 48 tokens**	**~8 – 10 tokens**
Fertility Score (Tokens / Word)	**~1.8 – 2.4** *(Good out-of-the-box)*	**~6.5 – 8.0** *(Poor without tokenizer extension)*	**~1.3 – 1.7** *(Ideal native level)*

What This Means for Your Architecture:

1. **If you DO NOT want to modify the tokenizer:** Choose **Gemma-2-2B** or **Gemma-3-1B / 4B**. Because its 256k+ vocabulary already includes Sinhala characters and common subwords, you can jump straight into **QLoRA Supervised Fine-Tuning (SFT)** without resizing the embedding matrices.
2. **If you want to use Qwen-2.5-1.5B:** Out of the box, Qwen-2.5-1.5B has terrible Sinhala fertility (~6.5+ tokens per word). However, researchers have already shown (such as the `iCIIT/general-purpose-model-ris-sinhala-qwen2.5-1.5b-cp` checkpoint on Hugging Face) that if you **extend Qwen's tokenizer with Sinhala tokens** and run continued pre-training, perplexity drops from `4.6` down to `3.05`.

2. Google Colab Script: Test Token Fertility & Compression Ratio

You do not need a GPU to test tokenizers—they run on CPU in seconds and only download a few megabytes (`tokenizer.json`). Paste this directly into a Google Colab cell:

```
# 1. Install Hugging Face Transformers
# !pip install -q transformers tabulate

import unicodedata
from transformers import AutoTokenizer
```

```
from tabulate import tabulate

# Note: For Gemma models, make sure you have accepted the license on Hugging Face
# and set your HF_TOKEN in Colab Secrets (or login via huggingface_hub.login())
MODELS_TO_TEST = {
    "Qwen-2.5-1.5B (Out-of-box)": "Qwen/Qwen2.5-1.5B-Instruct",
    "Qwen-2.5-1.5B (Sinhala-Extended)": "iCIIT/general-purpose-model-ris-sinhala-qwen2.5-1.5b-cp",
    # Uncomment the Gemma lines below once your HF_TOKEN is configured in Colab:
    # "Gemma-2-2B-IT": "google/gemma-2-2b-it",
    # "Gemma-3-4B-IT": "google/gemma-3-4b-it",
}

# Sample corpus covering Formal (Likhitha), Spoken (Kathitha), and Singlish Code-Mixing
TEST_SENTENCES = [
    # 1. Colloquial Spoken Sinhala (Kathitha)
    "මම අද කොළඹ යනවා, ඔයා හවසට එනවද මාව හම්බවෙන්න?",
    # 2. Formal Written Sinhala (Likhitha)
    "ශ්‍රී ලංකාවේ ආර්ථිකය සහ තොරතුරු තාක්ෂණ ක්ෂේත්‍රය පිළිබඳව විශේෂ අවධානයක් යොමු කළ යුතුය.",
    # 3. Conversational Customer Support / Technical
    "මගේ ගිණුමේ ශේෂය පරීක්ෂා කරගන්න පුළුවන් කොහොමද කියලා පැහැදිලි කරන්න.",
    # 4. Singlish + Sinhala Code-Mixing
    "Mata me API endpoint eka integrate karana widiha සිංහලෙන් explain karanna පුළුවන්ද?"
]

def evaluate_tokenizer(model_name, repo_id, sentences):
    print(f"\n{'='*70}")
    print(f"🔍 MODEL: {model_name} ({repo_id})")
    print(f"{'='*70}")

    try:
        tokenizer = AutoTokenizer.from_pretrained(repo_id, trust_remote_code=True)
    except Exception as e:
        print(f"⚠️ Could not load {repo_id}. (Check HF_TOKEN if gated): {e}")
        return None

    total_words = 0
    total_tokens = 0
    total_chars = 0

    for idx, text in enumerate(sentences, 1):
```

```
            # Always normalize Sinhala text to NFC before tokenizing
            clean_text = unicodedata.normalize("NFC", text)
            words = clean_text.split()

            # Encode without special BOS/EOS tokens for accurate fertility math
            token_ids = tokenizer.encode(clean_text, add_special_tokens=False)

            # Decode each token ID individually to see exact subword splits
            decoded_pieces = [tokenizer.decode([tid]) for tid in token_ids]

            num_words = len(words)
            num_tokens = len(token_ids)
            fertility = num_tokens / num_words

            total_words += num_words
            total_tokens += num_tokens
            total_chars += len(clean_text)

            if idx == 1:  # Print visual breakdown for the first sentence
                print(f"Sample Text   : {clean_text}")
                print(f"Decoded Pieces: {decoded_pieces}")
                print(f"Word Count    : {num_words} | Token Count: {num_tokens} | Fertility: {fertility:.2f} tokens/word")

        overall_fertility = total_tokens / total_words
        chars_per_token = total_chars / total_tokens

        return [
            model_name,
            f"{tokenizer.vocab_size:,}",
            total_words,
            total_tokens,
            f"{overall_fertility:.2f}",
            f"{chars_per_token:.2f}"
        ]

    # Run benchmark across all models
    summary_rows = []
    for name, repo in MODELS_TO_TEST.items():
        row = evaluate_tokenizer(name, repo, TEST_SENTENCES)
        if row:
```

```
        summary_rows.append(row)

print("\n\n📊 FINAL TOKEN FERTILITY BENCHMARK (Lower Fertility = Better/Faster)")
headers = ["Model", "Vocab Size", "Total Words", "Total Tokens", "Fertility (Tok/Word)", "Chars/Token"]
print(tabulate(summary_rows, headers=headers, tablefmt="github"))
```

3. How to Read Your Results

When you run the script above, look closely at two numbers in the final table:

1. **Fertility (`Tokens / Word`):**
    - **Less than `2.5`:** The model already understands the Sinhala script natively. You can fine-tune it directly using QLoRA on a free Colab T4 GPU.
    - **Greater than `4.0`:** The model is falling back to raw UTF-8 bytes. Not only will it generate text 3x slower, it will also struggle with Sinhala spelling and grammar unless you expand the tokenizer first.
2. **Decoded Pieces (`[tokenizer.decode([tid])]`):**
    - Look for the replacement character (`which happens when a multi-byte Sinhala character gets cut in half across two tokens). **Gemma 2 / 3** and **Sinhala-extended Qwen** will output clean readable Sinhala syllables in `Decoded Pieces`, whereas base Qwen/Llama will output` symbols for partial byte tokens.

User prompt: you are saying out of all iCIIT/general-purpose-model-ris-sinhala-qwen2.5-1.5b-cp is the best option?

Response: No—**`iCIIT/general-purpose-model-ris-sinhala-qwen2.5-1.5b-cp` is not the best overall option.** I included it in the benchmark script as an existing community baseline from the **iCIIT 2025 Shared Task** so you could compare how a custom Sinhala-adapted Qwen model behaves against standard out-of-the-box models.

If you look closely at how that specific checkpoint was built, it has several limitations that make **Gemma-2-2B** or **Gemma-3 (1B / 4B)** a much stronger foundation for your own project.

1. What `iCIIT/...-qwen2.5-1.5b-cp` Actually Is (And Its Limitations)

That model was a student/research submission for the iCIIT 2025 Shared Task on lightweight Sinhala LLMs. While it is a cool proof-of-concept that reduced Sinhala perplexity from `4.6` to `3.05`, relying on it as your primary base model has four major drawbacks:

- **Trained Only on Wikipedia (`-cp` = Continual Pre-training):** Its training corpus was just a cleaned mid-2024 Sinhala Wikipedia dump. Wikipedia is written in rigid, formal encyclopedic Sinhala (*Likhitha Bhashawa*), meaning the model has no natural conversational flow (*Kathitha Bhashawa*) and was not trained on modern multi-turn chat instructions.
- **Catastrophic Forgetting of Instruct Capabilities:** Because the authors took `Qwen-2.5-1.5B-Instruct` and ran full-parameter continual pre-training on raw Wikipedia articles for 3 epochs, the model overwrote much of its original instruction-following alignment. It acts more like a Wikipedia text-completer than a helpful assistant.
- **No Singlish or Code-Mixing:** The authors explicitly noted in their technical report that code-mixed Sinhala-English performance was not included or tested.
- **Bloated FP32 Weights (~6.2 GB):** The weights uploaded to that repository were saved in unquantized `F32` (32-bit float) across two safetensors files totaling over 6 GB—which is unnecessarily heavy for a 1.5B parameter model (which should only be ~3 GB in `BF16` or ~1 GB in 4-bit).

(Note: That same competition also produced `iCIIT/TripleBits-Sinhala-Gemma-2-2b-FT` and `iCIIT/TripleBits-Sinhala-Gemma-2-2b-CP`, which used the Gemma-2-2B architecture instead.)

2. So Which Model IS the Best Option for You?

Your "best" starting point depends entirely on whether you want **the best final model quality** or **the deepest academic research contribution**:

Goal / Scenario	Top Recommended Base Model	Why It Wins
Best Overall Quality & Fastest Path to a Working Sinhala Assistant	**`google/gemma-3-4b-it`** or **`google/gemma-2-2b-it`**	Google's 256k+ SentencePiece tokenizer already has native Sinhala tokens built in. You don't have to fight broken tokenizers or retrain embedding layers from scratch—you can jump straight into **QLoRA Supervised Fine-Tuning (SFT) + DPO** on conversational Sinhala and get a super fluent chat model in hours.
Best for Ultra-Light Offline / Edge / Mobile Deployment (~1B)	**`google/gemma-3-1b-it`**	At 1B parameters (~700 MB quantized), it retains the 262k vocabulary and native Sinhala script handling, making it far superior to `Llama-3.2-1B` or `Qwen-2.5-1.5B` out of the box.
Best If Your Research Thesis Specifically Focuses on "Tokenizer Surgery"	**`Qwen/Qwen2.5-1.5B`** or **`meta-llama/Llama-3.2-1B`**	Because Qwen and Llama have *bad* out-of-the-box Sinhala tokenizers, choosing them lets you write a research paper where *you* build a custom Sinhala morphological/grapheme tokenizer, merge it, and prove a 3x–4x speedup over the base model.

3. My Recommendation for Your Next Step

Do **not** build on top of someone else's half-finished shared-task checkpoint (`iCIIT/...`), because any quirks or catastrophic forgetting in their Wikipedia training run will bottleneck your model.

Instead:

1. **Start clean from `google/gemma-2-2b-it` (or `gemma-3-4b-it`).**
2. Put 80% of your effort into **curating a high-quality, novel Sinhala dataset** (mixing natural spoken Sinhala, formal Sinhala, and Romanized Singlish instructions) and running **QLoRA SFT + DPO preference alignment**.
3. Use `iCIIT/general-purpose-model-ris-sinhala-qwen2.5-1.5b-cp` and `polyglots/SinLlama_v01` purely as **competitor baselines in your evaluation table**—showing that your aligned Gemma model beats them on conversational fluency and instruction following!

Sources

1. iCIIT/general-purpose-model-ris-sinhala-qwen2.5-1.5b-cp
2. iCIIT Shared Task 2025 - Hugging Face

User prompt: I want to build a small sized, locally runnable on laptop or any desktop, stay by side like a friend, family person, special person or any instructor, teacher, like anything. the LLM is like just a actual person. it always ask or talk. it is not just a text based LLM also need voice as well. should talk like a real person. this is my goal for just now.

Response: To build a locally runnable AI companion that talks, listens, remembers you, and spontaneously initiates conversation like a real friend, family member, partner, or teacher, **an LLM alone is not enough**. A raw LLM only responds when typed to; to make it feel like a living person sitting beside you on your laptop, you need a **4-part local modular pipeline** that fits entirely inside **4 GB to 6 GB of RAM/VRAM**.

1. The Local Companion Architecture (Fits on Any Laptop)

Instead of one giant model, modern local voice companions run four lightweight models side-by-side. Every component below has native open-source Sinhala support and runs offline on a standard laptop CPU or entry-level GPU:

Human Organ	Technical Component	Best Local Models for Sinhala	RAM / VRAM Footprint
1. The Ears (Always Listening + Speech-to-Text)	**Silero VAD + Faster-Whisper (CTranslate2)**	• **VAD:** `snakers4/silero-vad` (detects when you start/stop speaking in 1ms)	**~400 MB – 600 MB**

Human Organ	Technical Component	Best Local Models for Sinhala	RAM / VRAM Footprint
		• **STT:** `SinhaSpeech/whisper-small-sinhala-run11` or `Lingalingeswaran/whisper-small-sinhala`	
2. The Brain (Persona & Conversational LLM)	**Ollama / llama.cpp** (4-bit GGUF)	• **Base:** `google/gemma-3-4b-it` or `gemma-2-2b-it` (or `Tharusha_Dilhara_Jayadeera/singemma` on Ollama) • **Alternative Multimodal:** `gemma-3n-E2B-it` (accepts raw audio directly without Whisper)	**~1.6 GB – 2.8 GB**
3. The Mouth (Natural Sinhala Voice Synthesis)	**VITS / Piper Local TTS**	• `dialoglk/SinhalaVITS-TTS-F1` (Expressive Female voice) • `SafnasKaldeen/Sinhala-Text-to-Speech-VITS-BestInSriLanka` (Trained on 4 native Sri Lankan voices: Isuru, Yasindu, Harini, Dinithi)	**~200 MB – 350 MB**
4. The Heart & Memory (Proactivity & Relationship State)	**Python Async Event Loop + Local Vector/SQLite DB**	• **ChromaDB / SQLite** (stores facts about your life, daily schedule, and past chats) • **APScheduler** (triggers spontaneous check-ins)	**< 100 MB**

2. How to Make It Act Like a Real Person (Not a Chatbot)

Standard AI assistants feel robotic because they wait silently for a command and speak in long, formal paragraphs. To make your Sinhala model feel like a real person sitting next to you, you must engineer four behavioral mechanics:

1. **Spontaneous Conversation ("The Heartbeat Loop"):** Real friends and family don't wait for you to prompt them. You run a background Python timer every 15–30 minutes that passes the **current time, silence duration, and recent memory** to the LLM with a hidden trigger `[SYSTEM_EVENT: User has been quiet for 40 minutes at 11:30 PM]`. The model then speaks out loud on its own:
    - *Friend / Special Person Persona: "ඒයි, දැන් ඇ 11:30 යි නේද? තාම වැඩද? පොඩ්ඩක් ඇස් දෙකට නිවාඩුවක් දෙන්න."* ("Hey, it's 11:30 PM right? Still working? Give your eyes a little rest.")

- *Teacher / Instructor Persona: "අපි කලින් කතා කරපු පාඩමේ ප්‍රශ්න ටික ඉවරද? මම පොඩි ප්‍රශ්නයක් අහන්නද?"* ("Did you finish the questions from the lesson we discussed earlier? Shall I ask a quick question?")

2. **Short, Spoken Colloquial Sinhala (*Kathitha Bhashawa*) + Filler Words:** Real people never speak in bullet points or formal written Sinhala (*Likhitha Bhashawa*). When you fine-tune your Gemma-2B/4B model, every assistant response in your training dataset must be **1 to 2 short spoken sentences** with natural Sri Lankan conversational fillers (*"හ්ම්ම්..."*, *"ආ ඇත්තද?"*, *"ඉතින් කියන්නකෝ"*, *"හරි හරි"*).

3. **Dynamic Persona Switching (Friend, Partner, Family, Teacher):** Instead of training four separate models, train **one model** using Role Tags in the system prompt (`[ROLE: BEST_FRIEND]`, `[ROLE: SPECIAL_PARTNER]`, `[ROLE: MOTHER_FIGURE]`, `[ROLE: TEACHER]`). That way, the tone, vocabulary, and affection level shift instantly depending on who you want beside you.

4. **Full-Duplex Barge-In (Interruptibility):** If the AI is explaining something and you suddenly say *"ආ නෑ නෑ, මම කිව්වේ ඒක නෙවෙයි"* ("Oh no no, that's not what I meant"), **Silero VAD** detects your voice within 30 milliseconds and immediately cuts off the TTS audio playback so the AI stops talking and listens to you—just like a human conversation.

3. Step-by-Step Roadmap to Build Your First Prototype

1. **Step 1: Get the Real-Time Voice Loop Working (1 Weekend)** Set up a local Python script using **Pipecat** or **RealtimeSTT + Ollama + VITS**. Connect your microphone to `SinhaSpeech/whisper-small-sinhala-run11` (converted to `faster-whisper`), feed the transcribed Sinhala text into `gemma-3-4b-it` running on **Ollama**, and stream the output sentence-by-sentence into `dialoglk/SinhalaVITS-TTS-F1` or `SafnasKaldeen/Sinhala-Text-to-Speech-VITS-BestInSriLanka`.

2. **Step 2: Add Memory & the Proactive Heartbeat** Add a lightweight local SQLite database that saves a 2-sentence summary at the end of every conversation (e.g., *"User has a software engineering interview on Thursday; felt stressed today"*). Connect a background timer so the model greets you and follows up on yesterday's events automatically when you open your laptop.

3. **Step 3: Fine-Tune the Brain for Spoken Emotional Sinhala (QLoRA)** Generate a synthetic dataset of **5,000–10,000 multi-turn spoken Sinhala dialogues** across your four personas (Friend, Family, Special Person, Teacher)—strictly enforcing short, warm, spoken-style responses and proactive follow-up questions—and fine-tune `gemma-2-2b-it` or `gemma-3-4b-it` using Unsloth QLoRA, then export to 4-bit GGUF for your laptop.

4. **Step 4: Train or Clone a Custom Expressive Voice (Optional Upgrade)** If existing Sinhala VITS models sound too much like a newsreader, use the open-source **`SafnasKaldeen/Sinhala-Text-speech-Dataset-TTS`** (which contains 10+ hours of studio-recorded male and female Sinhala voices) or record 30 minutes of a warm, conversational Sinhala voice to fine-tune **Piper TTS** or **F5-TTS**.

Would you like to start by building the **local Python voice-to-voice pipeline** (connecting Sinhala Whisper + Ollama + Sinhala VITS on your laptop), or by designing the **multi-persona spoken Sinhala dataset** to fine-tune the model's personality?
